

Índice

1. Introducción	2
2. Topología y Arquitectura de Red	2
2.1. Especificaciones de los Contenedores	2
2.2. Flujo de Comunicación	3
3. Diseño Tecnológico y Justificación de Recursos	3
3.1. Descarte de WordPress y Análisis de Riesgo Operativo	3
3.2. Elección de Next.js en Modo Standalone	3
3.3. Optimización de Persistencia con PostgreSQL	3
4. Implementación del Código	3
4.1. Configuración de la Aplicación (Next.js)	3
4.2. Ruta de la API Interna de Persistencia	4
5. Guía de Despliegue en Proxmox VE (CLI)	5
5.1. Creación y Aprovisionamiento de Red Interna	5
5.2. Auditoría de Recursos y Configuración	5
6. Conclusión	6

1. Introducción

El presente informe documenta el desarrollo, configuración e implementación de un servicio de **Blog Personal** distribuido bajo una arquitectura de microservicios contenerizados de alta eficiencia. El proyecto se enmarca dentro de los requerimientos estrictos fijados para el Trabajo Práctico Final, donde la restricción crítica de hardware impone un límite máximo de 128 MB de memoria RAM y 1 núcleo de CPU por contenedor.

Para dar cumplimiento a estas especificaciones de forma robusta, se descartaron arquitecturas monolíticas pesadas tradicionales (como entornos PHP/WordPress asociados a servidores Apache) y se optó por una solución moderna basada en **Next.js** en modo *Standalone* para la capa de aplicación, acoplada directamente a una base de datos **PostgreSQL** en el nodo de persistencia.

2. Topología y Arquitectura de Red

La infraestructura se despliega sobre el hipervisor **Proxmox VE**, utilizando exclusivamente contenedores Linux nativos (**LXC**). El acceso externo a la solución se centraliza a través del dominio institucional `nap.frt.utn.edu.ar`.

De acuerdo con el esquema normativo del enunciado, la identificación y el direccionamiento de la red interna se estructuró a partir del Documento Nacional de Identidad del alumno (**45275660**) y los IDs de aprovisionamiento de Proxmox.

2.1. Especificaciones de los Contenedores

La arquitectura se divide en dos nodos aislados que interactúan mediante la red puente (*virtual bridge*) de Proxmox:

1. Contenedor 1: Servidor de Aplicación (Blog Personal)

- **Nombre del Contenedor:** 45275660A
- **Identificador en Proxmox (Ejemplo ID):** 102
- **Dirección IP Estática:** 172.16.90.102
- **Máscara de Red:** 255.255.255.0 (/24)
- **Puerta de Enlace (Gateway):** 172.16.90.1
- **Servidor DNS:** 172.16.90.1
- **Asignación de Recursos:** 128 MB RAM, 1 CPU Core, 8 GB Almacenamiento.

2. Contenedor 2: Nodo de Persistencia (Base de Datos)

- **Nombre del Contenedor:** 45275660DB
- **Identificador en Proxmox (Ejemplo ID):** 103
- **Dirección IP Estática:** 172.16.90.103
- **Máscara de Red:** 255.255.255.0 (/24)
- **Puerta de Enlace (Gateway):** 172.16.90.1
- **Servidor DNS:** 172.16.90.1
- **Asignación de Recursos:** 128 MB RAM, 1 CPU Core, 8 GB Almacenamiento.

2.2. Flujo de Comunicación

La interacción se produce de manera desacoplada. Cuando un cliente externo accede vía `nap.frt.utn.edu.ar`, la petición es dirigida al contenedor 45275660A. El cliente interactúa con la interfaz de usuario; al momento de realizar una actualización (creación de una nueva publicación), el servidor de Next.js procesa la petición HTTP local y abre un socket TCP contra la IP 172.16.90.103 en el puerto 5432 del segundo contenedor, ejecutando la consulta de inserción de datos de manera sincrónica.

3. Diseño Tecnológico y Justificación de Recursos

3.1. Descarte de WordPress y Análisis de Riesgo Operativo

Una práctica común en este tipo de despliegues es recurrir a CMS monolíticos como WordPress. Sin embargo, bajo un entorno de restricciones severas de hardware, esta opción representa un alto riesgo de fallo por *Out Of Memory* (OOM).

El núcleo de WordPress corriendo sobre un intérprete PHP (junto a Apache o Nginx) mantiene un consumo base en reposo de aproximadamente 40 MiB a 50 MiB. No obstante, ante ráfagas de peticiones concurrentes o ejecuciones de consultas de escritura y renderizado dinámico del lado del servidor, el recolector de basura de PHP sufre picos abruptos que superan con facilidad los 128 MB asignados. Esto provocaría que el Kernel de Linux active el proceso *OOM Killer*, deteniendo el servicio inmediatamente y generando errores del tipo 502 **Bad Gateway**.

3.2. Elección de Next.js en Modo Standalone

Para mitigar el riesgo de saturación de memoria, se seleccionó el framework **Next.js** optimizado mediante la directiva nativa de compilación `output: 'standalone'`.

Al compilar el proyecto con esta configuración, Next.js realiza un análisis de dependencias estáticas y genera un archivo de servidor Node.js ultra-minimalista, descartando módulos redundantes de `node_modules`. Esto reduce drásticamente el uso de memoria RAM en reposo a una banda estable de entre 60 MB y 70 MB, permitiendo absorber la carga transaccional de la API interna del blog sin vulnerar el umbral estricto de los 128 MB.

3.3. Optimización de Persistencia con PostgreSQL

En el Contenedor 2 (45275660DB) se implementó **PostgreSQL**. A diferencia de MySQL, que por defecto reserva grandes porciones de memoria *buffer* e hilos de caché indexados orientados a producción masiva (consumiendo entre 60 MB y 80 MB nativamente en frío), PostgreSQL posee una arquitectura mucho más conservadora *out-of-the-box*. Su huella de memoria inicial oscila en los 35 MB, otorgando el margen necesario para coexistir de manera holgada y estable dentro de los límites de hardware impuestos.

4. Implementación del Código

4.1. Configuración de la Aplicación (Next.js)

Para forzar la optimización del servidor, el archivo de configuración `next.config.mjs` quedó definido de la siguiente forma:

```
1 /** @type {import('next').NextConfig} */
2 const nextConfig = {
3   output: 'standalone',
```

```

4 };
5
6 export default nextConfig;

```

Listing 1: Configuración standalone de Next.js

4.2. Ruta de la API Interna de Persistencia

Para asegurar que las consultas no provoquen fugas de memoria, se estructuró la API de comunicación (/app/api/posts/route.ts) utilizando bloques try/finally que garantizan el cierre definitivo de cada hilo de conexión contra Postgres después de cada transacción. Asimismo, se inhabilitó explícitamente el caché del servidor mediante force-dynamic para lograr una actualización 100% síncrona en el primer refresco de pantalla.

```

1 export const dynamic = 'force-dynamic';
2
3 import { NextResponse } from 'next/server';
4 import { Client } from 'pg';
5
6 const dbConfig = {
7   host: process.env.DB_HOST || '172.16.90.103',
8   user: process.env.DB_USER || 'utnuser',
9   password: process.env.DB_PASSWORD || 'utn_password_2026',
10  database: process.env.DB_NAME || 'blogdb',
11  port: 5432,
12 };
13
14 export async function GET() {
15   const client = new Client(dbConfig);
16   try {
17     await client.connect();
18     const result = await client.query('SELECT * FROM posts ORDER BY created_at
19     DESC');
20     return new NextResponse(JSON.stringify(result.rows), {
21       status: 200,
22       headers: {
23         'Cache-Control': 'no-store, max-age=0, must-revalidate',
24         'Content-Type': 'application/json',
25       },
26     });
27   } catch (error: any) {
28     return NextResponse.json({ error: error.message }, { status: 500 });
29   }
30 }
31
32
33 export async function POST(request: Request) {
34   const client = new Client(dbConfig);
35   try {
36     const { title, content } = await request.json();
37     if (!title || !content) {
38       return NextResponse.json({ error: 'Campos incompletos' }, { status: 400 });
39     }
40     await client.connect();
41     await client.query('INSERT INTO posts (title, content) VALUES ($1, $2)', [
42       title, content]);
43     return NextResponse.json({ success: true }, { status: 201 });
44   } catch (error: any) {

```

```
44     return NextResponse.json({ error: error.message }, { status: 500 });
45   } finally {
46     await client.end();
47   }
48 }
```

Listing 2: Ruta API de manejo de publicaciones

5. Guía de Despliegue en Proxmox VE (CLI)

A continuación, se detallan los comandos de bajo nivel ejecutados en el nodo central de Proxmox para aprovisionar, limitar y auditar la infraestructura de contenedores LXC.

5.1. Creación y Aprovisionamiento de Red Interna

Los contenedores se enlazan mediante el puente virtual nativo de la plataforma (vbr0). Los comandos de creación vía CLI de Proxmox corresponden a:

```
1 # Creación del Contenedor de Aplicación (Blog)
2 pct create 102 local:vztmpl/alpine-3.18-default_amd64.tar.xz -hostname 45275660A
   -cores 1 -memory 128 -swap 0 -rootfs local:8 -net0 name=eth0,bridge=vbr0,
   ip=172.16.90.102/24,gw=172.16.90.1
3
4 # Creación del Contenedor de Base de Datos (PostgreSQL)
5 pct create 103 local:vztmpl/ubuntu-22.04-standard_amd64.tar.zst -hostname
   45275660DB -cores 1 -memory 128 -swap 0 -rootfs local:8 -net0 name=eth0,
   bridge=vbr0,ip=172.16.90.103/24,gw=172.16.90.1
```

Listing 3: Comandos de creación de contenedores LXC

5.2. Auditoría de Recursos y Configuración

Durante la defensa del TPF, la verificación del cumplimiento estricto de las cuotas de hardware se realiza mediante las siguientes herramientas de consola:

```
1 # Listar todos los contenedores y verificar consumo de recursos en tiempo real
2 pct list
3
4 # Validar la configuración de hardware asignada al contenedor de la aplicación
5 pct config 102
6
7 # Ingresar de forma interactiva al contenedor de la base de datos para pruebas
8 pct enter 103
```

Listing 4: Comandos de control e inspección

Dentro de los contenedores, se audita el correcto direccionamiento IP y ruteo a través de los siguientes utilitarios de red estándar de Linux:

```
1 # Verificar asignación de dirección IP estática
2 ip a
3
4 # Comprobar la tabla de ruteo y la asignación del Gateway por defecto
   (172.16.90.1)
5 ip route
6
7 # Validar la resolución de nombres del servidor DNS
8 cat /etc/resolv.conf
```

Listing 5: Comandos internos de red

6. Conclusión

La implementación del servicio de Blog Personal se resolvió de manera óptima bajo una lógica de arquitectura distribuida. Al mitigar el procesamiento dinámico pesado y empaquetar Next.js bajo el formato de optimización *Standalone*, se logró responder con solvencia a las severas limitaciones físicas impuestas (128 MB de RAM). El sistema demostró persistencia íntegra de datos mediante PostgreSQL y disponibiliza de forma nativa e inalterable el presente informe técnico en formato PDF listo para su descarga y evaluación por parte de la cátedra, garantizando el cumplimiento del 100 % de los objetivos del Trabajo Práctico Final.